Sprint 8 Structured Notes: Branches, Pull Requests, and Code Review

1. Current Progress

You are now at **Sprint 8** of the Relational Databases learning path.

So far, you have learned:

- **Sprint 1:** VS Code and workspace setup
- **Sprint 2:** Terminal, shell, and command line basics
- **Sprint 3:** Bash file and folder commands
- **Sprint 4:** Bash scripting basics
- **Sprint 5:** Nano and Git setup
- **Sprint 6:** Local Git workflow
- **Sprint 7:** GitHub repositories and remotes
- **Sprint 8:** Branches, Pull Requests, and Code Review

Simple meaning:

You are still inside the **Git section** of the course. You are **not learning PostgreSQL or SQL yet**. Before databases, you must first become comfortable with how developers manage and share code safely.

In Sprint 7, you learned how to connect a local repository to GitHub using:

- `origin`
- `git push`
- `git pull`
- `git remote -v`

Now in Sprint 8, you learn how developers work safely without putting every change directly into the `main` branch.

---

2. Main Goal of Sprint 8

The main goal of Sprint 8 is:

Work on isolated changes and merge them safely through a pull request.

Simple meaning:

In Sprint 8, you learn how to:

- create a separate branch
- make changes there
- push that branch to GitHub
- open a pull request
- review the changes
- merge them safely into `main`

This is one of the most common real-world Git workflows.

---

3. Big Beginner Idea

Think about Sprint 8 like this:

```
main branch = clean final project
feature branch = separate working copy
pull request = request to add your work into main
code review = checking before accepting the work
merge = adding the work into main
```

Important beginner rule:

Do **not** treat `main` like a place for random testing and unfinished experiments.

Simple example:

```
main = final clean notebook
feature branch = rough draft paper
pull request = asking someone to check the rough draft
merge = adding the approved draft into the final notebook
```

Simple meaning:

Sprint 8 teaches you how to keep the main project stable while still allowing new work to be added safely.

---

4. What Is a Branch?

A **branch** is a separate path in your Git project history.

Simple meaning:

A branch lets you work on something without changing the main project directly.

Example:

```
main
 └── feature
```

Here:

```
main = stable project
feature = new work
```

You use a branch when you want to:

- add a new file
- fix a bug
- test an idea
- make a small feature
- update documentation

Simple meaning:

A branch is a safe separate work area.

---

5. What Is the `main` Branch?

The `main` branch is usually the main stable branch of the project.

Simple meaning:

`main` should contain clean, accepted, working project history.

You should not directly push unfinished work to `main`.

Better workflow:

```
main → create feature branch → work → commit → push branch → pull request → merge
```

Simple meaning:

Use `main` as the stable home of the project, not as the place for every unfinished change.

---

6. What Is a Checked-Out Branch?

The **checked-out branch** is the branch you are currently using.

To check your current branch, run:

```
git branch
```

Example output:

```
* main
  feature
```

The `*` means:

```
You are currently on this branch.
```

So in the example above, you are currently on `main`.

Simple meaning:

The checked-out branch is the branch your terminal is currently working on.

---

7. What Is a Feature Branch?

A **feature branch** is a branch created for one specific task or change.

Example branch names:

```
feature
feature/sprint8-note
fix/readme-typo
docs/update-readme
```

For beginners, this is okay:

```
git checkout -b feature
```

A more descriptive real-world example:

```
git checkout -b feature/sprint8-note
```

Simple meaning:

A feature branch is a safe place to work on one task.

---

8. What Is a Pull Request?

A **pull request**, often called a **PR**, is a request to merge one branch into another branch.

Usually:

```
feature branch → main branch
```

Simple meaning:

A pull request asks:

Can we add these changes into `main`?

In GitHub, a pull request usually shows:

- what changed
- which files changed
- who made the change
- which branch the change comes from
- which branch the change will merge into

Simple meaning:

A pull request is the review step before adding new work into the main project.

---

9. Base Branch and Compare Branch

When you create a pull request on GitHub, GitHub asks you for two branches.

Base branch

This is where the changes will go.

Usually:

```
main
```

Simple meaning:

The **base branch** is the target branch.

Compare branch / head branch

This is the branch that contains your new work.

Example:

```
feature
```

Simple meaning:

The **compare branch** is the branch you want to merge.

So this means:

```
Merge feature into main
```

Simple meaning:

Base = destination

Compare = source of the new work

---

10. What Is a Diff View?

A **diff** shows the exact changes between two branches or two versions of a file.

GitHub often shows lines like this:

```
+ added line
- removed line
```

Simple meaning:

The diff view shows exactly what changed before you merge.

Before merging a pull request, always check the diff.

Ask yourself:

```
Did I add only the files I wanted?
Did I accidentally commit private files?
Is my change clear?
Is the code or text correct?
```

Simple meaning:

Do not merge without checking what changed.

---

11. What Is Code Review?

Code review means another person checks your pull request before it is merged.

They may:

- approve it
- ask questions
- request changes
- suggest edits
- merge it

Simple meaning:

Code review is a safety check before your work enters the main project.

Even when working alone, you should still review your own pull request carefully.

Simple meaning:

Code review helps catch mistakes before they become part of the stable branch.

---

12. Merge, Squash, and Rebase Merge

When a pull request is ready, GitHub may show different merge options.

12.1 Normal Merge

A normal merge adds the branch into `main` and keeps the branch history.

Simple meaning:

Keep the commits and merge them into `main`.

12.2 Squash and Merge

This combines all commits from the branch into one commit.

Simple meaning:

Many small commits become one clean commit.

This is useful when your branch has messy commits like:

```
update
fix
try again
final fix
```

12.3 Rebase and Merge

This places your commits on top of the latest `main` history.

Simple meaning:

It makes history look more straight and clean.

Beginner note:

For Sprint 8 practice, the normal merge flow is the easiest place to start.

---

13. Sprint 8 Full Practice Workflow

Use your Sprint 7 GitHub practice repository.

Step 1: Go into your project folder

```
cd sprint7-practice
```

Step 2: Check your branches

```
git branch
```

Step 3: Go to `main`

```
git checkout main
```

Step 4: Get the latest GitHub version

```
git pull
```

Step 5: Create a new feature branch

```
git checkout -b feature
```

Or using modern Git:

```
git switch -c feature
```

Step 6: Create a new file

```
echo "This is our new feature" > feature.md
```

Step 7: Check status

```
git status
```

Step 8: Stage the file

```
git add feature.md
```

Step 9: Commit the change

```
git commit -m "feat: add feature note"
```

Step 10: Push the branch to GitHub

```
git push -u origin feature
```

Now go to GitHub in your browser.

You may see a button like:

```
Compare & pull request
```

Step 11: Open the pull request

Set:

```
base: main
compare: feature
```

Step 12: Add a PR title

Example:

```
Add Sprint 8 feature note
```

Step 13: Add a PR description

Example:

```
This PR adds a feature.md file for Sprint 8 branch and pull request practice.
```

Step 14: Review the diff

Go to **Files changed** and check the changes carefully.

Step 15: Merge the pull request

Merge the PR on GitHub.

Step 16: Update local `main`

After merging on GitHub, come back to your terminal and run:

```
git checkout main
git pull
```

Simple meaning:

Now your local `main` branch has the merged change.

---

14. Sprint 8 Command Summary

`git branch`

```
git branch
```

Shows branches.

`git branch feature`

```
git branch feature
```

Creates a branch called `feature`.

`git checkout feature`

```
git checkout feature
```

Switches to the `feature` branch.

`git switch feature`

```
git switch feature
```

Modern way to switch to the `feature` branch.

`git checkout -b feature`

```
git checkout -b feature
```

Creates and switches to `feature`.

`git switch -c feature`

```
git switch -c feature
```

Modern way to create and switch to `feature`.

`git push -u origin feature`

```
git push -u origin feature
```

Pushes the feature branch to GitHub and remembers the upstream connection.

`git checkout main` and `git pull`

```
git checkout main
git pull
```

Returns to `main` and downloads the latest merged changes.

---

15. Why `-u` Matters in `git push -u origin feature`

The `-u` flag means Git will remember the connection between your local branch and the remote branch.

Example:

```
git push -u origin feature
```

Simple meaning:

The first time, you connect your local `feature` branch to the remote `feature` branch.

After that, future pushes are easier.

---

16. Common Beginner Mistakes

Mistake 1: Working directly on `main`

Wrong habit:

```
git checkout main
# edit files directly
git push
```

Better habit:

```
git checkout main
git pull
git checkout -b feature
```

Simple meaning:

Create a separate branch before doing new work.

Mistake 2: Forgetting to push the branch

A local branch stays only on your computer until you push it.

Use:

```
git push -u origin feature
```

Mistake 3: Creating the pull request in the wrong direction

Correct:

```
base: main
compare: feature
```

Wrong:

```
base: feature
compare: main
```

Simple meaning:

You want to merge `feature` into `main`, not `main` into `feature`.

Mistake 4: Not checking the diff

Always check **Files changed** before merging.

Simple meaning:

Do not merge blindly.

Mistake 5: Forgetting to pull after merging

After merging on GitHub, your local `main` is not automatically updated.

Run:

```
git checkout main
git pull
```

Simple meaning:

Merge on GitHub first, then update your local project.

---

17. Sprint 8 Checkpoint

Your Sprint 8 checkpoint is:

Create and merge one pull request in your own practice repository.

You pass Sprint 8 when you can do this:

```
git checkout main
git pull
git checkout -b feature
echo "This is our new feature" > feature.md
git add feature.md
git commit -m "feat: add feature note"
git push -u origin feature
```

Then on GitHub:

```
Open pull request
Review diff
Merge pull request
```

Then locally:

```
git checkout main
git pull
```

Simple meaning:

If you can do this full flow, you understand the main idea of Sprint 8.

---

18. Mini Assessment

Answer these before moving to Sprint 9:

1. What is a branch?
2. Why should we not work directly on `main`?
3. What is a feature branch?
4. What does `git branch` show?
5. What does `git checkout -b feature` do?
6. What is a pull request?
7. What is the base branch?
8. What is the compare branch?
9. What is a diff view?
10. What is code review?
11. What does `git push -u origin feature` do?
12. What should you do locally after merging a PR on GitHub?

---

19. Final Sprint 8 Understanding

After Sprint 8, you should understand that Git work is not only about commits and pushing to GitHub. You should now understand:

- what a branch is
- why `main` should stay stable
- what a feature branch is
- what a pull request is
- what base and compare branches mean
- what a diff view shows

- what code review means
- what merge options mean
- how to create a branch and push it
- how to open a pull request
- how to merge safely
- how to update local `main` after merging

Best simple summary:

Sprint 8 teaches me to create a separate branch, make a change there, push that branch to GitHub, open a pull request, review the changes, merge safely into `main`, and then pull the updated `main` branch locally.